

Documentação PowerShell para Testar um Microserviço em Go

1. Objetivo

O PowerShell pode ser usado como cliente para testar uma API REST criada em Go.

No nosso microserviço, temos endpoints como:

```
GET /produtos
GET /produtos/{id}
POST /produtos
```

O endereço do servidor é:

```
http://localhost:8080
```

Antes de qualquer teste, execute o microserviço:

```
go run .
```

Se tudo estiver correto:

```
Servidor rodando em http://localhost:8080
```

2. Invoke-RestMethod

O comando mais importante para nossos testes será:

```
Invoke-RestMethod
```

Ele serve para realizar requisições HTTP para APIs REST.

Podemos utilizá-lo para:

```
GET
POST
PUT
PATCH
DELETE
```

Uma vantagem é que ele entende JSON automaticamente.

3. Estrutura básica

A estrutura mais simples é:

```
Invoke-RestMethod -Uri "ENDereco" -Method METODO
```

Exemplo:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method GET
```

Temos:

Invoke-RestMethod
Comando que realiza a chamada HTTP.

-Uri
Endereço que queremos acessar.

-Method
Método HTTP utilizado.

4. GET: listar produtos

Para consultar todos os produtos:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method GET
```

Estamos chamando:

GET /produtos

O servidor poderá responder:

```
[
  {
    "id": 1,
    "nome": "Notebook",
    "preco": 3500
  },
  {
    "id": 2,
    "nome": "Mouse",
    "preco": 100
  }
]
```

5. GET: buscar um produto específico

Para buscar o produto de ID 1:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos/1" `
    -Method GET
```

Resposta:

```
{
  "id": 1,
  "nome": "Notebook",
  "preco": 3500
}
```

Para buscar o produto 2:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos/2" `
    -Method GET
```

6. Guardando a resposta em uma variável

Podemos armazenar a resposta da API.

```
$produtos = Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method GET
```

Agora:

```
$produtos
```

mostra os produtos.

Também podemos acessar campos específicos.

```
$produtos[0].nome
```

Resultado:

```
Notebook
```

Ou:

```
$produtos[0].preco
```

Resultado:

```
3500
```

7. POST: criar um produto

O POST é usado para enviar informações e criar um recurso.

Exemplo:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method POST `
    -ContentType "application/json" `
    -Body '{"nome":"Webcam","preco":299.90}'
```

Estamos enviando:

```
{
  "nome": "Webcam",
  "preco": 299.90
}
```

O servidor poderá responder:

```
{
  "id": 3,
  "nome": "Webcam",
  "preco": 299.9
}
```

8. O parâmetro -Body

O parâmetro:

```
-Body
```

representa o corpo da requisição HTTP.

Exemplo:

```
-Body '{ "nome": "Webcam", "preco": 299.90 } '
```

É nele que enviamos os dados para a API.

Fluxo:

```
PowerShell
```

↓

```
Body
```

↓

```
JSON
```

↓

```
Microserviço Go
```

↓

```
Controller
```

No Go, esses dados são recebidos por:

```
json.NewDecoder(r.Body).Decode(&produto)
```

9. O parâmetro -ContentType

Quando enviamos JSON, devemos informar:

```
-ContentType "application/json"
```

Isso cria o cabeçalho HTTP:

```
Content-Type: application/json
```

Basicamente estamos dizendo ao servidor:

```
"Os dados que estou enviando estão em JSON."
```

10. Uma maneira melhor de criar JSON no PowerShell

Em vez de escrever manualmente:

```
'{ "nome": "Webcam", "preco": 299.90 } '
```

podemos criar um objeto PowerShell.

```
$produto = @{  
    nome = "Webcam"  
    preco = 299.90  
}
```

Depois transformamos em JSON:

```
$json = $produto | ConvertTo-Json
```

Podemos verificar:

```
$json
```

Resultado:

```
{  
  "preco": 299.9,  
  "nome": "Webcam"  
}
```

Agora enviamos:

```
Invoke-RestMethod `  
    -Uri "http://localhost:8080/produtos" `  
    -Method POST `  
    -ContentType "application/json" `  
    -Body $json
```

Essa é uma forma muito segura de trabalhar com JSON no PowerShell.

11. ConvertTo-Json

O comando:

```
ConvertTo-Json
```

transforma um objeto PowerShell em JSON.

Exemplo:

```
$produto = @{  
    nome = "Monitor"  
    preco = 1200  
}
```

Depois:

```
$produto | ConvertTo-Json
```

Resultado:

```
{  
  "nome": "Monitor",  
  "preco": 1200  
}
```

Podemos pensar assim:

```
Objeto PowerShell  
    ↓  
ConvertTo-Json
```

JSON

↓

12. Exemplo completo de POST

Podemos fazer:

```
$produto = @{
    nome = "Teclado"
    preco = 250
}

$json = $produto | ConvertTo-Json

Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method POST `
    -ContentType "application/json" `
    -Body $json
```

Essa forma é especialmente interessante para aulas porque separa claramente as etapas.

13. O caractere ` no PowerShell

Observe:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method GET
```

O caractere:

`

é chamado de:

backtick

No PowerShell ele indica que o comando continua na próxima linha.

Assim:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method GET
```

é equivalente a:

```
Invoke-RestMethod -Uri "http://localhost:8080/produtos" -Method GET
```

Usamos várias linhas apenas para facilitar a leitura.

14. Invoke-WebRequest

Outro comando do PowerShell é:

```
Invoke-WebRequest
```

Exemplo:

```
Invoke-WebRequest `
  -Uri "http://localhost:8080/produtos" `
  -Method GET
```

Ele também realiza requisições HTTP.

A principal diferença didática é:

```
Invoke-RestMethod
```

Mais conveniente para APIs e JSON.

```
Invoke-WebRequest
```

Mais genérico e útil quando queremos analisar a resposta HTTP completa.

15. Verificando o status HTTP

Com `Invoke-WebRequest` podemos observar melhor detalhes da resposta.

```
$resposta = Invoke-WebRequest `
  -Uri "http://localhost:8080/produtos" `
  -Method GET
```

Depois:

```
$resposta.StatusCode
```

Resultado:

```
200
```

HTTP 200 significa:

```
OK
```

16. Alguns códigos HTTP importantes

Durante os testes do microserviço podemos encontrar:

```
200 OK
```

Requisição realizada com sucesso.

```
201 Created
```

Um recurso foi criado com sucesso.

```
400 Bad Request
```

A requisição enviada está incorreta.

```
404 Not Found
```

O recurso solicitado não foi encontrado.

```
405 Method Not Allowed
```

O método HTTP utilizado não é permitido.

```
500 Internal Server Error
```

Erro interno no servidor.

Por exemplo, ao criar um produto, nosso Go retorna:

```
w.WriteHeader(http.StatusCreated)
```

Isso representa:

```
201 Created
```

17. curl.exe no PowerShell

O Windows também possui o programa:

```
curl.exe
```

Podemos listar produtos com:

```
curl.exe "http://localhost:8080/produtos"
```

Ou explicitamente:

```
curl.exe -X GET "http://localhost:8080/produtos"
```

18. POST usando curl.exe

Para criar um produto:

```
curl.exe -X POST "http://localhost:8080/produtos" `
  -H "Content-Type: application/json" `
  -d "{\"nome\":\"Webcam\", \"preco\":299.90}"
```

Aqui:

```
-X POST
```

Define o método HTTP.

```
-H
```

Adiciona um cabeçalho HTTP.

```
-d
```

Define os dados enviados.

19. Invoke-RestMethod ou curl.exe?

Para aulas utilizando PowerShell, uma opção bastante simples é:

```
Invoke-RestMethod
```

Exemplo:

```
Invoke-RestMethod `
  -Uri "http://localhost:8080/produtos" `
  -Method GET
```

Ele possui integração melhor com objetos do PowerShell.

Já o:

```
curl.exe
```

é muito utilizado porque existe também em outros sistemas operacionais, como Linux e macOS.

20. Testando nosso microserviço passo a passo

Primeiro execute o servidor:

```
go run .
```

Em outro terminal PowerShell:

Teste 1: listar produtos

```
Invoke-RestMethod `
  -Uri "http://localhost:8080/produtos" `
  -Method GET
```

Teste 2: buscar produto

```
Invoke-RestMethod `
  -Uri "http://localhost:8080/produtos/1" `
  -Method GET
```

Teste 3: criar produto

```
$produto = @{
  nome = "Webcam"
  preco = 299.90
}
```

```
$json = $produto | ConvertTo-Json
```

```
Invoke-RestMethod `
  -Uri "http://localhost:8080/produtos" `
  -Method POST `
  -ContentType "application/json" `
  -Body $json
```

Teste 4: listar novamente

```
Invoke-RestMethod `
  -Uri "http://localhost:8080/produtos" `
  -Method GET
```

Agora a Webcam deverá aparecer na lista.

21. Resumo dos comandos

Os principais comandos utilizados serão:

```
go run .
```

Inicia nosso microserviço Go.

```
Invoke-RestMethod
```

Realiza chamadas para APIs REST.

```
Invoke-WebRequest
```

Realiza requisições HTTP e permite analisar mais detalhes da resposta.

ConvertTo-Json

Transforma objetos PowerShell em JSON.

curl.exe

Programa de linha de comando utilizado para realizar requisições HTTP.

22. Fluxo completo

Quando executamos:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method GET
```

o fluxo é:

```
PowerShell
↓
Invoke-RestMethod
↓
HTTP GET
↓
localhost:8080
↓
Microserviço Go
↓
main.go
↓
Controller
↓
Service
↓
Produtos
↓
Controller
↓
JSON
↓
PowerShell
```

No caso de POST:

```
PowerShell
↓
Objeto
↓
ConvertTo-Json
↓
JSON
↓
Invoke-RestMethod
↓
POST /produtos
↓
Microserviço Go
↓
Controller
↓
```

Service
↓
Novo produto

Regra prática

Para trabalhar com nosso microserviço no PowerShell, podemos guardar quatro comandos principais:

GET

```
Invoke-RestMethod -Uri "URL" -Method GET
```

POST

```
Invoke-RestMethod -Uri "URL" -Method POST -ContentType "application/json" -Body $json
```

Objeto → JSON

```
$objeto | ConvertTo-Json
```

Ver detalhes HTTP

```
Invoke-WebRequest
```

Com esses comandos já conseguimos testar praticamente toda a API REST exemplo de nosso curso